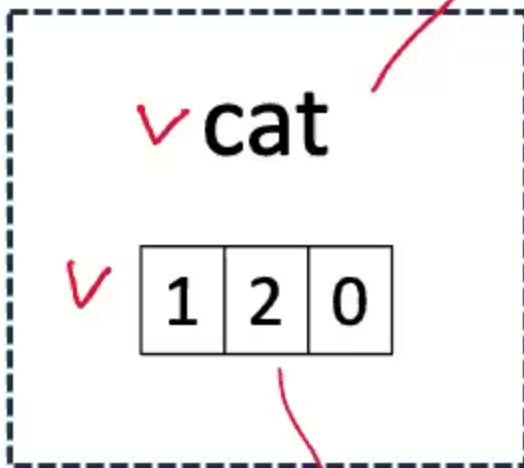


Which record is relevant to the query “cat”?

2 modi:

- per nome; il più simile a «cat» è forse «bat» (una sola lettera di differenza)
- per embedding: vorrei trovare «dog»!

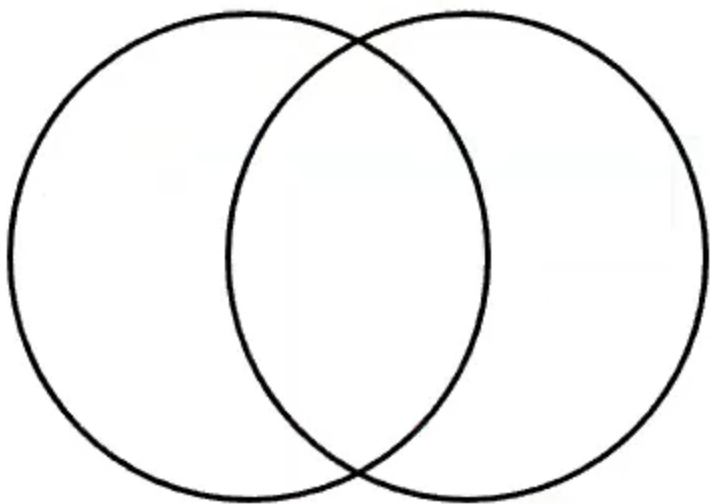
Query



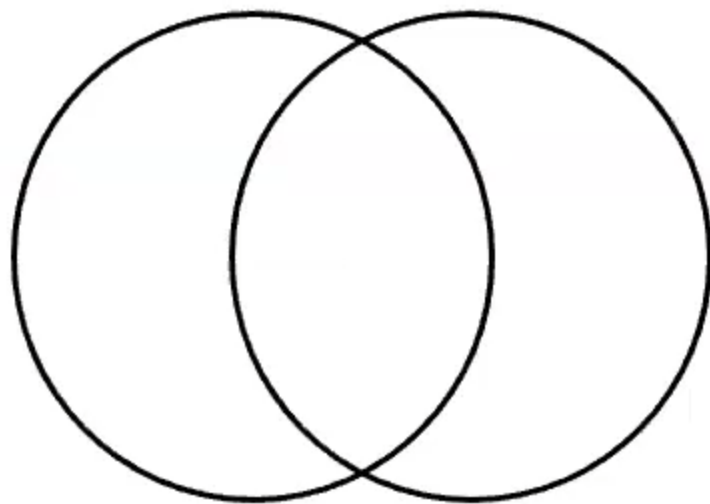
id	name	size	pop	emb			
1	dog	2	900	<table><tr><td>2</td><td>1</td><td>0</td></tr></table>	2	1	0
2	1	0					
2	bat	1	10000	<table><tr><td>0</td><td>1</td><td>2</td></tr></table>	0	1	2
0	1	2					

Draw distance vs similarity

distance

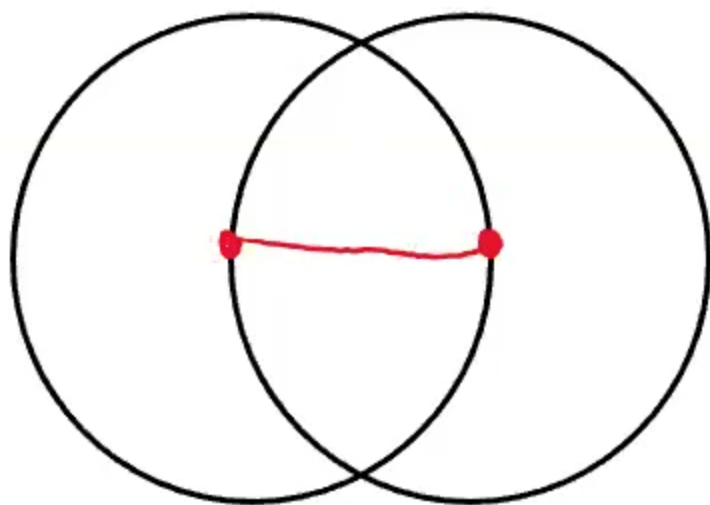


similarity

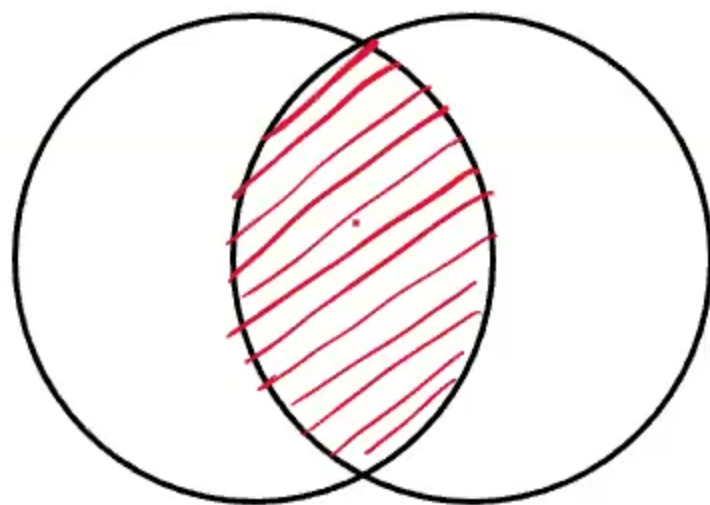


Draw distance vs similarity

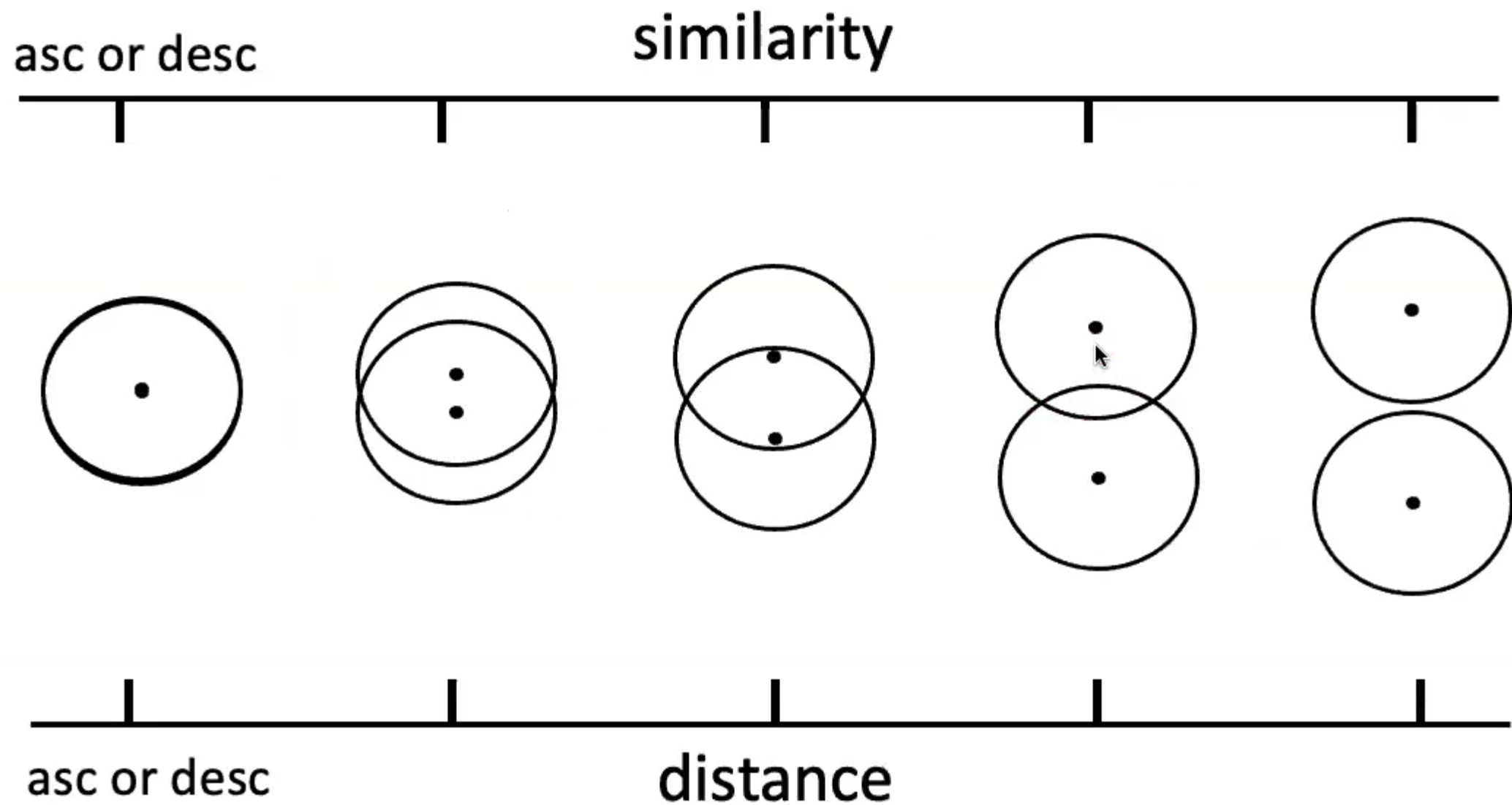
distance



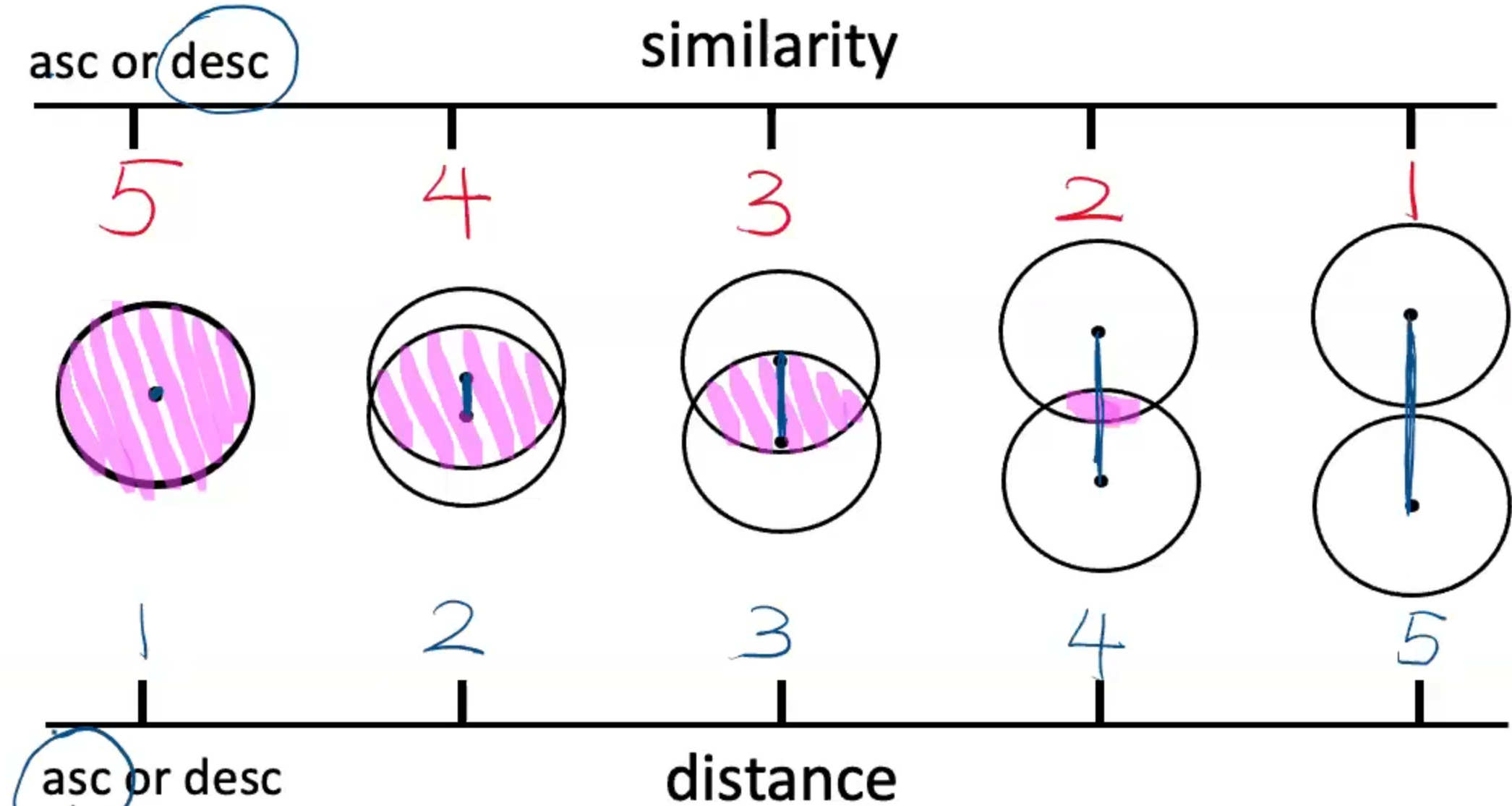
similarity



Distance vs similarity on a scale of 1 to 5

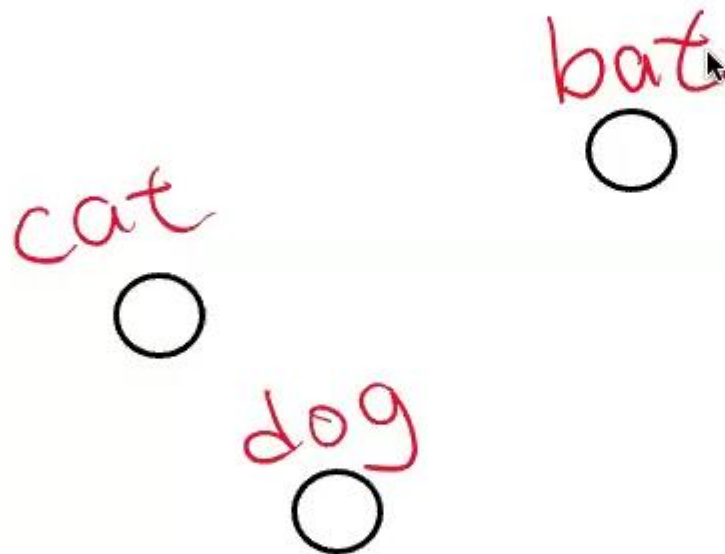


Distance vs similarity on a scale of 1 to 5



Where are dog, cat and bat in the “~~name~~”
space?

Semantic



In uno spazio di questo tipo – il significato delle parole – *cat* e *dog* sono vicini perché entrambi animali domestici

Which embedding is better?

Embedding 1

dog	cat	bat
2	1	0
1	2	1
0	0	2

Embedding 2

dog	cat	bat
2	0	1
1	1	0
0	2	2

Supponiamo di avere due schemi di embedding: qual è il migliore in questo caso?

Which embedding is better?

Desired
dot
product
similarity

la matrice

dog	cat	bat
2	1	0
1	2	1
0	0	2

Embedding 1

dog	cat	bat
2	1	0
1	2	1
0	0	2

Embedding 2

dog	cat	bat
2	0	1
1	1	0
0	2	2

dog	2	1	0
cat	1	2	0
bat	0	1	2

la matrice trasposta

	H	L
H		L
L	L	

la matrice di
somiglianza

(simmetrica) VOLUTA

dog	2	1	0
cat	1	2	0
bat	0	1	2

dog	2	1	0
cat	0	1	2
bat	1	0	2

Due matrici da confrontare: quale è più simile a quella desiderata?

Which embedding is better?

Desired
dot
product
similarity

dog	cat	bat
2	1	0
1	2	1
0	0	2

Embedding 1

dog	cat	bat
2	1	0
1	2	1
0	0	2

Embedding 2

dog	cat	bat
2	0	1
1	1	0
0	2	2

dog	2	1	0
cat	1	2	0
bat	0	1	2

	H	L
H		
L		

dog	2	1	0
cat	1	2	0
bat	0	1	2

	4	1
4		
1		

dog	2	1	0
cat	0	1	2
bat	1	0	2

	1	2
1		
2		

Ci sono vari algoritmi di ML che, data una serie di parole, cercano in modo iterativo i loro embedding «migliori», cioè tali che la similarità tra le parole sia la più simile possibile a quella

la matrice
migliore

E' una spiegazione intuitiva ad alto livello.

Ci sono vari algoritmi di ML che, data una serie di parole, cercano in modo iterativo i loro embedding «migliori», cioè tali che la similarità tra le parole sia la più simile possibile a quella desiderata.

Il processo descritto è molto costoso, nellapratica ci si affida a siti come *Hugging Face*, dal quale si possono scaricare ottimi schemi di embedding già ottimizzati per vari tipi di vocabolario. Anche OpenAI ha i suoi schemi di embedding.

How to embed sentences?

id	comment	user	emb
1	How are you?	John	?
2	Who are you?	Mary	?

Come codificare frasi, anziché parole? In modo poi da poterle confrontare tra loro?

“How are you” → word embedding vectors

how	are	you
-----	-----	-----

Questi embedding sono stati precedentemente trovati con un algoritmo di ML iterativo, cioè sino al punto nel quale i loro prodotti interni a coppie (cioè le similarità semantiche a coppie) erano massime per questo vocabolario: le parole simili avevano i prodotti interni più alti

a	an	the	how	why	who	what	are	is	am	be	was	you	we	I	they	she	he	she	me	him	her
0	-1	0	1	0	1	0	0	-1	1	0	0	0	3	1	0	-1	0	0	0	-1	0
2	0	2	0	0	0	-1	1	0	0	0	2	1	0	2	0	2	0	0	2	0	0
-1	0	-1	1	2	0	0	1	0	1	-1	0	0	-1	0	3	0	0	-1	0	2	-1
0	1	0	0	1	0	1	0	1	0	1	-2	0	0	0	1	0	1	0	1	0	1

“How are you” → word embedding vectors

how are you

a	an	the	how	why	who	what	are	is	am	be	was	you	we	I	they	she	he	she	me	him	her
0	-1	0	1	0	1	0	0	-1	1	0	0	0	3	1	0	-1	0	0	0	-1	0
2	0	2	0	0	0	-1	1	0	0	0	2	1	0	2	0	2	0	0	2	0	0
-1	0	-1	1	2	0	0	1	0	1	-1	0	0	-1	0	3	0	0	-1	0	2	-1
0	1	0	0	1	0	1	0	1	0	1	-2	0	0	0	1	0	1	0	1	0	1

1	0	0
0	1	1
1	1	0
0	0	0

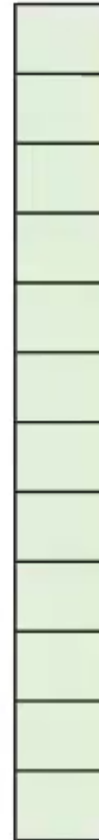
Gli embedding vector
delle tre parole della
frase in oggetto

Nel DB si memorizza il sentence vector, non i word vector in genere.

Word vectors $\rightarrow$ Sentence vector

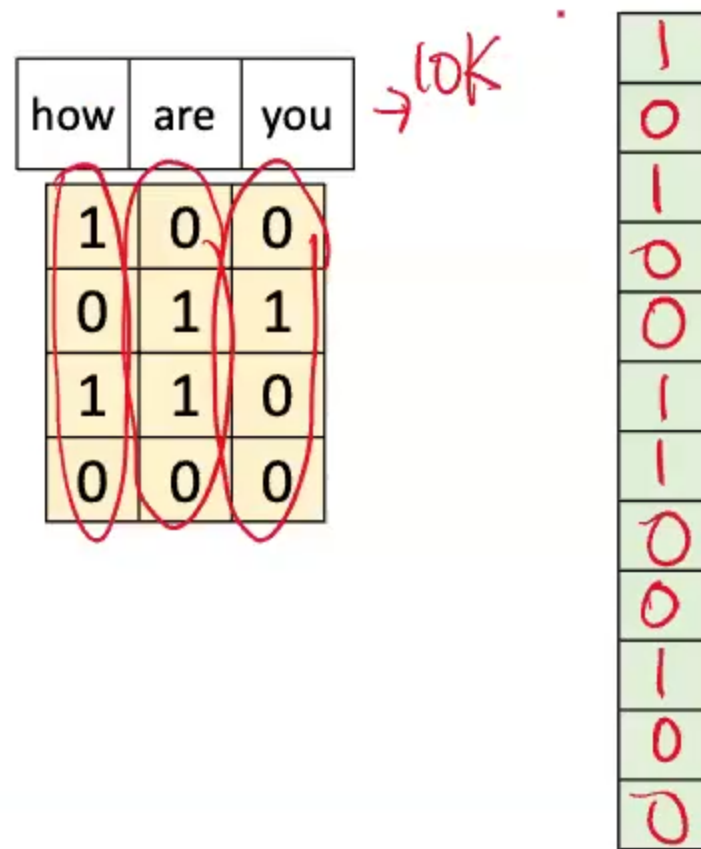
Method 1: Concatenate

how	are	you
1	0	0
0	1	1
1	1	0
0	0	0



Word vectors $\rightarrow$ Sentence vector

Method 1: Concatenate



Il metodo di concatenazione può funzionare con piccoli frasi, ma scala male con frasi lunghe $\rightarrow$ il vettore cresce troppo di lunghezza.

Word vectors $\rightarrow$ Sentence vector

Method 2: Average

how	are	you
1	0	0
0	1	1
1	1	0
0	0	0

id	comment	user	emb
1	How are you?	John	
2	Who are you?	Mary	

E' il metodo in genere preferito, crea vettori di frase della stessa dimensione dei vettori parola.

Word vectors $\rightarrow$ Sentence vector

Method 2: Average

how	are	you
-----	-----	-----

1	0	0
0	1	1
1	1	0
0	0	0

$\frac{1}{3}$
$\frac{2}{3}$
$\frac{2}{3}$
$\frac{0}{3}$

VECTOR(4)

id	comment	user	emb
1	How are you?	John	$[\frac{1}{3}, \frac{2}{3}, \frac{2}{3}, \frac{0}{3}]$
2	Who are you?	Mary	

“Who are you” → word embedding vectors

who	are	you
-----	-----	-----

a	an	the	how	why	who	what	are	is	am	be	was	you	we	I	they	she	he	she	me	him	her
0	-1	0	1	0	1	0	0	-1	1	0	0	0	3	1	0	-1	0	0	0	-1	0
2	0	2	0	0	0	-1	1	0	0	0	2	1	0	2	0	2	0	0	2	0	0
-1	0	-1	1	2	0	0	1	0	1	-1	0	0	-1	0	3	0	0	-1	0	2	-1
0	1	0	0	1	0	1	0	1	0	1	-2	0	0	0	1	0	1	0	1	0	1

	0	0
	1	1
	1	0
	0	0

Solo una parola è nuova («who»), per le altre avevamo già estratto i due embedding vector

Word vectors $\rightarrow$ Sentence vector

Method 2: Average

who	are	you
1	0	0
0	1	1
0	1	0
0	0	0

$1/3$
$2/3$
$1/3$
$0/3$

Un processo di ML converte tutte le frasi (i «comment») nei rispettivi *sentence vector* («emb») e li memorizza in un DB molto grande

id	comment	user	emb
1	How are you?	John	$[1/3, 2/3, 2/3, 0]$
2	Who are you?	Mary	$[1/3, 2/3, 1/3, 0]$